Lista dwukierunkowa

Wygenerowano za pomocą Doxygen 1.18.0

1 Struktura katalogów **1**

1.1 Katalogi . . . 1

2 Indeks klas **3**

2.1 Lista klas . . . 3

3 Indeks plików **5**

3.1 Lista plików . . . 5

4 Dokumentacja katalogów **7**

4.1 Dokumentacja katalogu Lista_Dwukierunkowa . . . 7

5 Dokumentacja klas **9**

5.1 Dokumentacja klasy List . . . 9

5.1.1 Opis szczegółowy . . . 10

5.1.2 Dokumentacja konstruktora i destruktora . . . 10

5.1.2.1 List() . . . 10

5.1.2.2 ~List() . . . 10

5.1.3 Dokumentacja funkcji składowych . . . 11

5.1.3.1 addAtIndex() . . . 11

5.1.3.2 addBack() . . . 11

5.1.3.3 addFront() . . . 12

5.1.3.4 clear() . . . 12

5.1.3.5 display() . . . 12

5.1.3.6 displayReverse() . . . 12

5.1.3.7 getIterator() . . . 13

5.1.3.8 removeAtIndex() . . . 13

5.1.3.9 removeBack() . . . 13

5.1.3.10 removeFront() . . . 14

5.1.4 Dokumentacja atrybutów składowych . . . 14

5.1.4.1 head . . . 14

5.1.4.2 tail . . . 14

5.2 Dokumentacja klasy ListIterator . . . 14

5.2.1 Opis szczegółowy . . . 15

5.2.2 Dokumentacja konstruktora i destruktora . . . 15

5.2.2.1 ListIterator() . . . 15

5.2.3 Dokumentacja funkcji składowych . . . 15

5.2.3.1 displayCurrent() . . . 15

5.2.3.2 next() . . . 16

5.2.3.3 prev() . . . 16

5.2.4 Dokumentacja atrybutów składowych . . . 16

5.2.4.1 current . . . 16

5.3 Dokumentacja klasy Node . . . 16

5.3.1 Opis szczegółowy . . . 17

5.3.2 Dokumentacja konstruktora i destruktora . . . 17
5.3.2.1 Node() . . . 17
5.3.3 Dokumentacja atrybutów składowych . . . 17
5.3.3.1 data . . . 17
5.3.3.2 next . . . 17
5.3.3.3 prev . . . 18
5.4 Dokumentacja klasy NodeFactory . . . 18
5.4.1 Opis szczegółowy . . . 18
5.4.2 Dokumentacja funkcji składowych . . . 18
5.4.2.1 createNode() . . . 18

6 Dokumentacja plików **21**

6.1 Dokumentacja pliku Lista_Dwukierunkowa/List.cpp . . . 21
6.2 Dokumentacja pliku Lista_Dwukierunkowa/List.h . . . 21
6.2.1 Opis szczegółowy . . . 21
6.2.2 Dokumentacja definicji . . . 22
6.2.2.1 LIST_H . . . 22
6.3 List.h . . . 22
6.4 Dokumentacja pliku Lista_Dwukierunkowa/ListIterator.cpp . . . 22
6.4.1 Opis szczegółowy . . . 22
6.5 Dokumentacja pliku Lista_Dwukierunkowa/ListIterator.h . . . 23
6.5.1 Opis szczegółowy . . . 23
6.5.2 Dokumentacja definicji . . . 23
6.5.2.1 LISTITERATOR_H . . . 23
6.6 ListIterator.h . . . 23
6.7 Dokumentacja pliku Lista_Dwukierunkowa/main.cpp . . . 24
6.7.1 Dokumentacja funkcji . . . 24
6.7.1.1 main() . . . 24
6.8 Dokumentacja pliku Lista_Dwukierunkowa/Node.h . . . 25
6.8.1 Opis szczegółowy . . . 25
6.8.2 Dokumentacja definicji . . . 25
6.8.2.1 NODE_H . . . 25
6.9 Node.h . . . 25
6.10 Dokumentacja pliku Lista_Dwukierunkowa/NodeFactory.cpp . . . 26
6.10.1 Opis szczegółowy . . . 26
6.11 Dokumentacja pliku Lista_Dwukierunkowa/NodeFactory.h . . . 26
6.11.1 Opis szczegółowy . . . 26
6.11.2 Dokumentacja definicji . . . 26
6.11.2.1 NODEFACTORY_H . . . 26
6.12 NodeFactory.h . . . 26

Skorowidz **27**

Rozdział 1

Struktura katalogów

1.1 Katalogi

Lista_Dwukierunkowa 7
List.cpp 21
List.h 21
ListIterator.cpp 22
ListIterator.h 23
main.cpp 24
Node.h 25
NodeFactory.cpp 26
NodeFactory.h 26

Rozdział 2

Indeks klas

2.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

List
Implementuje dwukierunkową listę elementów typu int 9
ListIterator
Iterator umożliwiający przechodzenie po liście dwukierunkowej 14
Node
Reprezentuje pojedynczy węzeł listy dwukierunkowej 16
NodeFactory
Fabryka służąca do tworzenia węzłów listy 18

Rozdział 3

Indeks plików

3.1 Lista plików

Tutaj znajduje się lista wszystkich plików wraz z ich krótkimi opisami:

Lista_Dwukierunkowa/List.cpp 21
Lista_Dwukierunkowa/List.h
    Deklaracja klasy dwukierunkowej listy 21
Lista_Dwukierunkowa/ListIterator.cpp
    Implementacja iteratora listy dwukierunkowej 22
Lista_Dwukierunkowa/ListIterator.h
    Deklaracja iteratora listy dwukierunkowej 23
Lista_Dwukierunkowa/main.cpp 24
Lista_Dwukierunkowa/Node.h
    Deklaracja węzła listy dwukierunkowej 25
Lista_Dwukierunkowa/NodeFactory.cpp
    Implementacja fabryki węzłów listy dwukierunkowej 26
Lista_Dwukierunkowa/NodeFactory.h
    Deklaracja fabryki węzłów listy dwukierunkowej 26

Rozdział 4

Dokumentacja katalogów

4.1 Dokumentacja katalogu Lista_Dwukierunkowa

Pliki

- plik List.cpp
- plik List.h

  Deklaracja klasy dwukierunkowej listy.
- plik ListIterator.cpp

  Implementacja iteratora listy dwukierunkowej.
- plik ListIterator.h

  Deklaracja iteratora listy dwukierunkowej.
- plik main.cpp
- plik Node.h

  Deklaracja węzła listy dwukierunkowej.
- plik NodeFactory.cpp

  Implementacja fabryki węzłów listy dwukierunkowej.
- plik NodeFactory.h

  Deklaracja fabryki węzłów listy dwukierunkowej.

Rozdział 5

Dokumentacja klas

5.1 Dokumentacja klasy List

Implementuje dwukierunkową listę elementów typu int.

`#include <List.h>`

Metody publiczne

- List ()

  Konstruktor klasy List.
- ~List ()

  Destruktor klasy List.
- ListIterator getIterator ()

  Zwraca iterator rozpoczynający przechodzenie od początku listy.
- void addFront (int val)

  Dodaje element na początku listy.
- void addBack (int val)

  Dodaje element na końcu listy.
- void display ()

  Wyświetla wszystkie elementy listy od początku do końca.
- void addAtIndex (int index, int val)

  Wstawia element pod wskazanym indeksem.
- void removeFront ()

  Usuwa pierwszy element listy.
- void removeBack ()

  Usuwa ostatni element listy.
- void removeAtIndex (int index)

  Usuwa element znajdujący się pod wskazanym indeksem.
- void clear ()

  Usuwa wszystkie elementy listy.
- void displayReverse ()

  Wyświetla elementy listy w odwrotnej kolejności.

Atrybuty prywatne

- Node * head

  Wskaźnik na pierwszy element listy.
- Node * tail

  Wskaźnik na ostatni element listy.

5.1.1 Opis szczegółowy

Implementuje dwukierunkową listę elementów typu int.

Lista umożliwia dodawanie, usuwanie i wyświetlanie elementów, pobieranie iteratora oraz usuwanie wszystkich elementów. Każdy węzeł zawiera odwołania do poprzedniego i następnego elementu listy.

5.1.2 Dokumentacja konstruktora i destruktora

5.1.2.1 List()

```
List::List ()
```

Konstruktor klasy List.

Tworzy pustą listę i inicjalizuje wskaźniki head oraz tail.

Inicjalizuje pustą listę dwukierunkową poprzez ustawienie wskaźników head i tail na nullptr.

5.1.2.2 ~List()

```
List::~List ()
```

Destruktor klasy List.

Zwalnia pamięć zajmowaną przez wszystkie węzły listy.

Zwalnia pamięć zajmowaną przez wszystkie węzły listy poprzez wywołanie metody clear().

5.1.3 Dokumentacja funkcji składowych

5.1.3.1 addAtIndex()

```
void List::addAtIndex (
            int index,
            int val)
```

Wstawia element pod wskazanym indeksem.

Wstawia nowy element pod wskazanym indeksem.

Jeśli indeks jest mniejszy lub równy zero, element zostaje dodany na początku listy. Jeśli indeks wskazuje pozycję poza istniejącymi elementami, element zostaje dodany na końcu listy.

Parametry

index	Indeks pozycji, na której ma zostać wstawiony element.
val	Wartość całkowita dodawanego elementu.

Jeśli indeks jest mniejszy lub równy zero, element zostaje dodany na początku listy. Jeśli wskazana pozycja wykracza poza istniejące elementy, element zostaje dodany na końcu. W przeciwnym razie nowy węzeł zostaje wstawiony przed elementem znajdującym się pod wskazanym indeksem.

Parametry

index	Indeks pozycji, na której ma zostać wstawiony element.
val	Wartość całkowita nowego elementu.

5.1.3.2 addBack()

```
void List::addBack (
            int val)
```

Dodaje element na końcu listy.

Dodaje nowy element na koniec listy.

Parametry

val	Wartość całkowita dodawanego elementu.

Tworzy nowy węzeł za pomocą NodeFactory. Jeśli lista jest pusta, nowy węzeł staje się zarówno początkiem, jak i końcem listy. W przeciwnym razie zostaje dołączony za ostatnim węzłem.

Parametry

val	Wartość całkowita dodawanego elementu.

5.1.3.3 addFront()

```
void List::addFront (
            int val)
```

Dodaje element na początku listy.

Dodaje nowy element na początek listy.

Parametry

val	Wartość całkowita dodawanego elementu.

Tworzy nowy węzeł za pomocą NodeFactory. Jeśli lista jest pusta, nowy węzeł staje się zarówno pierwszym, jak i ostatnim elementem. W przeciwnym razie zostaje połączony z dotychczasowym początkiem listy.

Parametry

val	Wartość całkowita dodawanego elementu.

5.1.3.4 clear()

```
void List::clear ()
```

Usuwa wszystkie elementy listy.

Po wykonaniu operacji lista jest pusta, a pamięć zajmowana przez jej węzły zostaje zwolniona.

Kolejno usuwa pierwszy węzeł za pomocą metody removeFront(), aż lista stanie się pusta. Po zakończeniu wskaźniki head i tail mają wartość nullptr.

5.1.3.5 display()

```
void List::display ()
```

Wyświetla wszystkie elementy listy od początku do końca.

Jeśli lista jest pusta, wyświetla odpowiedni komunikat.

Jeśli lista jest pusta, wyświetla odpowiedni komunikat. W przeciwnym razie przechodzi kolejno przez węzły, rozpoczynając od wskaźnika head, i wypisuje ich wartości.

5.1.3.6 displayReverse()

```
void List::displayReverse ()
```

Wyświetla elementy listy w odwrotnej kolejności.

Rozpoczyna od ostatniego elementu i przechodzi w kierunku początku listy.

Rozpoczyna od ostatniego węzła wskazywanego przez tail i przechodzi w kierunku początku listy, korzystając ze wskaźników prev. Jeśli lista jest pusta, metoda kończy działanie bez wyświetlania elementów.

5.1.3.7 getIterator()

```
ListIterator List::getIterator ()
```

Zwraca iterator rozpoczynający przechodzenie od początku listy.

Tworzy iterator rozpoczynający przechodzenie od początku listy.

Zwraca

ListIterator Iterator wskazujący na pierwszy element listy.

Zwraca nowy obiekt iteratora zainicjalizowany wskaźnikiem na pierwszy węzeł listy.

Zwraca

ListIterator Iterator wskazujący na początek listy.

5.1.3.8 removeAtIndex()

```
void List::removeAtIndex (
            int index)
```

Usuwa element znajdujący się pod wskazanym indeksem.

Usuwa element listy znajdujący się pod wskazanym indeksem.

Jeśli indeks wynosi zero, usuwa pierwszy element. Jeśli indeks wskazuje ostatni element, usuwa ostatni węzeł. Nie usuwa elementu, jeśli wskazany indeks nie istnieje.

Parametry

index	Indeks elementu przeznaczonego do usunięcia.

Jeśli lista jest pusta lub indeks nie wskazuje istniejącego elementu, metoda nie usuwa żadnego węzła. Element o indeksie zero jest usuwany za pomocą removeFront(), a ostatni element za pomocą removeBack(). W pozostałych przypadkach sąsiadujące węzły zostają połączone ze sobą, a usuwany węzeł zwolniony.

Parametry

index	Indeks elementu przeznaczonego do usunięcia.

5.1.3.9 removeBack()

```
void List::removeBack ()
```

Usuwa ostatni element listy.

Jeśli lista jest pusta, metoda nie wykonuje żadnej operacji.

Jeśli lista nie jest pusta, usuwa ostatni węzeł i aktualizuje wskaźnik tail. Jeśli lista zawierała tylko jeden element, wskaźnik head również zostaje ustawiony na nullptr. Pamięć węzła zostaje zwolniona.

5.1.3.10 removeFront()

```
void List::removeFront ()
```

Usuwa pierwszy element listy.

Jeśli lista jest pusta, metoda nie wykonuje żadnej operacji.

Jeśli lista nie jest pusta, usuwa pierwszy węzeł i aktualizuje wskaźnik head. W przypadku usunięcia ostatniego elementu ustawia również tail na nullptr. Pamięć zajmowana przez usunięty węzeł zostaje zwolniona.

5.1.4 Dokumentacja atrybutów składowych

5.1.4.1 head

```
Node* List::head  [private]
```

Wskaźnik na pierwszy element listy.

W przypadku pustej listy przyjmuje wartość nullptr.

5.1.4.2 tail

```
Node* List::tail  [private]
```

Wskaźnik na ostatni element listy.

W przypadku pustej listy przyjmuje wartość nullptr.

Dokumentacja dla tej klasy została wygenerowana z plików:

- Lista_Dwukierunkowa/List.h
- Lista_Dwukierunkowa/List.cpp

5.2 Dokumentacja klasy ListIterator

Iterator umożliwiający przechodzenie po liście dwukierunkowej.

```
#include <ListIterator.h>
```

Metody publiczne

- ListIterator (Node *startNode)

  Konstruktor iteratora.
- void next ()

  Przesuwa iterator do następnego elementu listy.
- void prev ()

  Przesuwa iterator do poprzedniego elementu listy.
- void displayCurrent ()

  Wyświetla wartość aktualnego elementu listy.

Atrybuty prywatne

- Node * current

  Wskaźnik na aktualny węzeł listy.

5.2.1 Opis szczegółowy

Iterator umożliwiający przechodzenie po liście dwukierunkowej.

Iterator przechowuje wskaźnik na aktualny węzeł listy i udostępnia metody pozwalające zmieniać jego pozycję oraz wyświetlać wartość bieżącego elementu.

5.2.2 Dokumentacja konstruktora i destruktora

5.2.2.1 ListIterator()

```
ListIterator::ListIterator (
            Node * startNode)
```

Konstruktor iteratora.

Inicjalizuje iterator, ustawiając go na wskazanym węźle.

Parametry

startNode	Wskaźnik na węzeł, od którego rozpoczyna się iteracja.

Ustawia iterator na węźle przekazanym jako argument.

Parametry

startNode	Wskaźnik na węzeł, od którego rozpoczyna się iteracja. Może mieć wartość nullptr.

5.2.3 Dokumentacja funkcji składowych

5.2.3.1 displayCurrent()

```
void ListIterator::displayCurrent ()
```

Wyświetla wartość aktualnego elementu listy.

Sposób zachowania dla iteratora wskazującego nullptr zależy od implementacji metody.

Jeśli iterator wskazuje na istniejący węzeł, wyświetla przechowywaną w nim wartość. W przeciwnym razie wyświetla komunikat informujący o braku elementu.

5.2.3.2 next()

```
void ListIterator::next ()
```

Przesuwa iterator do następnego elementu listy.

Przechodzi do węzła znajdującego się za aktualnym elementem.

Jeśli iterator wskazuje na istniejący węzeł, zostaje przesunięty do następnego węzła za pomocą wskaźnika next. Jeśli current ma wartość nullptr, metoda nie wykonuje żadnej operacji.

5.2.3.3 prev()

```
void ListIterator::prev ()
```

Przesuwa iterator do poprzedniego elementu listy.

Przechodzi do węzła znajdującego się przed aktualnym elementem.

Jeśli iterator wskazuje na istniejący węzeł, zostaje przesunięty do poprzedniego węzła za pomocą wskaźnika prev. Jeśli current ma wartość nullptr, metoda nie wykonuje żadnej operacji.

5.2.4 Dokumentacja atrybutów składowych

5.2.4.1 current

```
Node* ListIterator::current  [private]
```

Wskaźnik na aktualny węzeł listy.

Wskazuje element, na którym aktualnie znajduje się iterator.

Dokumentacja dla tej klasy została wygenerowana z plików:

- Lista_Dwukierunkowa/ListIterator.h
- Lista_Dwukierunkowa/ListIterator.cpp

5.3 Dokumentacja klasy Node

Reprezentuje pojedynczy węzeł listy dwukierunkowej.

```
#include <Node.h>
```

Metody publiczne

- Node (int val)

  Konstruktor węzła.

Atrybuty publiczne

- int data

  Wartość przechowywana w węźle.
- Node ∗ next

  Wskaźnik na następny węzeł listy.
- Node ∗ prev

  Wskaźnik na poprzedni węzeł listy.

5.3.1 Opis szczegółowy

Reprezentuje pojedynczy węzeł listy dwukierunkowej.

Węzeł przechowuje wartość typu int oraz wskaźniki na następny i poprzedni węzeł listy.

5.3.2 Dokumentacja konstruktora i destruktora

5.3.2.1 Node()

```
Node::Node (
            int val)  [inline]
```

Konstruktor węzła.

Inicjalizuje węzeł podaną wartością oraz ustawia wskaźniki next i prev na nullptr.

Parametry

val	Wartość całkowita przechowywana w węźle.

5.3.3 Dokumentacja atrybutów składowych

5.3.3.1 data

```
int Node::data
```

Wartość przechowywana w węźle.

5.3.3.2 next

```
Node* Node::next
```

Wskaźnik na następny węzeł listy.

Wartość nullptr oznacza, że węzeł jest ostatnim elementem listy.

5.3.3.3 prev

```
Node* Node::prev
```

Wskaźnik na poprzedni węzeł listy.

Wartość nullptr oznacza, że węzeł jest pierwszym elementem listy.

Dokumentacja dla tej klasy została wygenerowana z pliku:

- Lista_Dwukierunkowa/Node.h

5.4 Dokumentacja klasy NodeFactory

Fabryka służąca do tworzenia węzłów listy.

```
#include <NodeFactory.h>
```

Statyczne metody publiczne

- static Node * createNode (int val)
  Tworzy nowy węzeł listy.

5.4.1 Opis szczegółowy

Fabryka służąca do tworzenia węzłów listy.

Klasa udostępnia statyczną metodę createNode(), która tworzy nowy węzeł i zwraca wskaźnik na niego.

5.4.2 Dokumentacja funkcji składowych

5.4.2.1 createNode()

```
Node * NodeFactory::createNode (
            int val)  [static]
```

Tworzy nowy węzeł listy.

Alokuje pamięć dla nowego obiektu Node i inicjalizuje go przekazaną wartością.

Parametry

val	Wartość całkowita przechowywana w węźle.

Zwraca

Node∗ Wskaźnik na utworzony węzeł.

Alokuje pamięć dla nowego obiektu klasy Node, inicjalizowanego przekazaną wartością.

Parametry

val	Wartość całkowita, która zostanie zapisana w węźle.

Zwraca

Node∗ Wskaźnik na nowo utworzony węzeł.

Nota

Pamięć jest alokowana za pomocą operatora new. Za zwolnienie pamięci odpowiada kod zarządzający listą.

Dokumentacja dla tej klasy została wygenerowana z plików:

- Lista_Dwukierunkowa/NodeFactory.h
- Lista_Dwukierunkowa/NodeFactory.cpp

Rozdział 6

Dokumentacja plików

6.1 Dokumentacja pliku Lista_Dwukierunkowa/List.cpp

```
#include <iostream>
#include "List.h"
#include "NodeFactory.h"
```

6.2 Dokumentacja pliku Lista_Dwukierunkowa/List.h

Deklaracja klasy dwukierunkowej listy.

```
#include "Node.h"
#include "ListIterator.h"
```

Komponenty

- class List

  Implementuje dwukierunkową listę elementów typu int.

Definicje

- #define LIST_H

6.2.1 Opis szczegółowy

Deklaracja klasy dwukierunkowej listy.

Plik zawiera deklarację klasy List, która umożliwia przechowywanie elementów typu int w postaci listy dwukierunkowej oraz wykonywanie operacji na jej elementach.

6.2.2 Dokumentacja definicji

6.2.2.1 LIST_H

```
#define LIST_H
```

6.3 List.h

Idź do dokumentacji tego pliku.

```
00001
00010
00011 #pragma once
00012
00013 #ifndef LIST_H
00014 #define LIST_H
00015
00016 #include "Node.h"
00017 #include "ListIterator.h"
00018
00028 class List {
00029 private:
00035     Node* head;
00036
00042     Node* tail;
00043
00044 public:
00050     List();
00051
00057     ~List();
00058
00064     ListIterator getIterator();
00065
00071     void addFront(int val);
00072
00078     void addBack(int val);
00079
00085     void display();
00086
00098     void addAtIndex(int index, int val);
00099
00105     void removeFront();
00106
00112     void removeBack();
00113
00123     void removeAtIndex(int index);
00124
00131     void clear();
00132
00139     void displayReverse();
00140 };
00141
00142 #endif
```

6.4 Dokumentacja pliku Lista_Dwukierunkowa/ListIterator.cpp

Implementacja iteratora listy dwukierunkowej.

```
#include "ListIterator.h"
#include <iostream>
```

6.4.1 Opis szczegółowy

Implementacja iteratora listy dwukierunkowej.

Plik zawiera implementację metod klasy ListIterator, które umożliwiają poruszanie się po liście w obu kierunkach oraz wyświetlanie wartości aktualnego elementu.

6.5 Dokumentacja pliku Lista_Dwukierunkowa/ListIterator.h

Deklaracja iteratora listy dwukierunkowej.

```
#include "node.h"
```

Komponenty

- class ListIterator

  Iterator umożliwiający przechodzenie po liście dwukierunkowej.

Definicje

- #define LISTITERATOR_H

6.5.1 Opis szczegółowy

Deklaracja iteratora listy dwukierunkowej.

Plik zawiera definicję klasy ListIterator, która umożliwia poruszanie się po elementach listy dwukierunkowej w przód i w tył oraz wyświetlanie aktualnego elementu.

6.5.2 Dokumentacja definicji

6.5.2.1 LISTITERATOR_H

```
#define LISTITERATOR_H
```

6.6 ListIterator.h

Idź do dokumentacji tego pliku.

```
00001
00010
00011 #pragma once
00012 #ifndef LISTITERATOR_H
00013 #define LISTITERATOR_H
00014
00015 #include "node.h"
00016
00025 class ListIterator {
00026 private:
00032     Node* current;
00033
00034 public:
00043     ListIterator(Node* startNode);
00044
00050     void next();
00051
00057     void prev();
00058
00065     void displayCurrent();
00066 };
00067
00068 #endif
```

6.7 Dokumentacja pliku Lista_Dwukierunkowa/main.cpp

```
#include <iostream>
#include "List.h"
```

Funkcje

- int main ()

  Funkcja główna programu.

6.7.1 Dokumentacja funkcji

6.7.1.1 main()

```
int main ()
```

Funkcja główna programu.

Demonstruje działanie listy, w tym dodawanie i usuwanie elementów, wyświetlanie listy w obu kierunkach, obsługę iteratora oraz czyszczenie listy.

Zwraca

int Zwraca 0, jeśli program zakończy się poprawnie.

Utworzenie obiektu listy.

Dodanie elementów na koniec listy.

Dodanie elementów na początek listy.

Wstawienie elementu 15 pod indeksem 1.

Wyświetlenie listy: 30 5 15 10 20 25.

Usunięcie elementu znajdującego się pod indeksem 1.

Wyświetlenie listy po usunięciu elementu: 30 15 10 20 25.

Wyświetlenie elementów listy w odwrotnej kolejności: 25 20 10 15 30.

Pobranie iteratora umożliwiającego przechodzenie po liście.

Wyświetlenie aktualnego elementu iteratora.

Przejście iteratora do następnego elementu i jego wyświetlenie.

Usunięcie wszystkich elementów listy.

Wyświetlenie komunikatu informującego o pustej liście.

6.8 Dokumentacja pliku Lista_Dwukierunkowa/Node.h

Deklaracja węzła listy dwukierunkowej.

Komponenty

- class Node

  Reprezentuje pojedynczy węzeł listy dwukierunkowej.

Definicje

- #define NODE_H

6.8.1 Opis szczegółowy

Deklaracja węzła listy dwukierunkowej.

Plik zawiera definicję klasy Node, która reprezentuje pojedynczy element listy dwukierunkowej.

6.8.2 Dokumentacja definicji

6.8.2.1 NODE_H

```
#define NODE_H
```

6.9 Node.h

Idź do dokumentacji tego pliku.

```
00001
00009
00010 #pragma once
00011
00012 #ifndef NODE_H
00013 #define NODE_H
00014
00022 class Node {
00023 public:
00027     int data;
00028
00035     Node* next;
00036
00043     Node* prev;
00044
00053     Node(int val) {
00054         data = val;
00055         next = nullptr;
00056         prev = nullptr;
00057     }
00058 };
00059
00060 #endif
```

6.10 Dokumentacja pliku Lista_Dwukierunkowa/NodeFactory.cpp

Implementacja fabryki węzłów listy dwukierunkowej.

```
#include "NodeFactory.h"
```

6.10.1 Opis szczegółowy

Implementacja fabryki węzłów listy dwukierunkowej.

Plik zawiera implementację metody tworzącej nowe węzły klasy Node.

6.11 Dokumentacja pliku Lista_Dwukierunkowa/NodeFactory.h

Deklaracja fabryki węzłów listy dwukierunkowej.

```
#include "node.h"
```

Komponenty

- class NodeFactory

  Fabryka służąca do tworzenia węzłów listy.

Definicje

- #define NODEFACTORY_H

6.11.1 Opis szczegółowy

Deklaracja fabryki węzłów listy dwukierunkowej.

Plik zawiera deklarację klasy NodeFactory, która udostępnia statyczną metodę tworzenia nowych węzłów klasy Node.

6.11.2 Dokumentacja definicji

6.11.2.1 NODEFACTORY_H

```
#define NODEFACTORY_H
```

6.12 NodeFactory.h

Idź do dokumentacji tego pliku.

```
00001
00009
00010 #pragma once
00011 #ifndef NODEFACTORY_H
00012 #define NODEFACTORY_H
00013
00014 #include "node.h"
00015
00023 class NodeFactory {
00024 public:
00034     static Node* createNode(int val);
00035 };
00036
00037 #endif
```

Skorowidz

~List
    List, 10

addAtIndex
    List, 11
addBack
    List, 11
addFront
    List, 11

clear
    List, 12
createNode
    NodeFactory, 18
current
    ListIterator, 16

data
    Node, 17
display
    List, 12
displayCurrent
    ListIterator, 15
displayReverse
    List, 12
Dokumentacja katalogu Lista_Dwukierunkowa, 7

getIterator
    List, 12

head
    List, 14

List, 9
    ~List, 10
    addAtIndex, 11
    addBack, 11
    addFront, 11
    clear, 12
    display, 12
    displayReverse, 12
    getIterator, 12
    head, 14
    List, 10
    removeAtIndex, 13
    removeBack, 13
    removeFront, 13
    tail, 14
List.h
    LIST_H, 22
LIST_H
    List.h, 22
Lista_Dwukierunkowa/List.cpp, 21
Lista_Dwukierunkowa/List.h, 21, 22
Lista_Dwukierunkowa/ListIterator.cpp, 22
Lista_Dwukierunkowa/ListIterator.h, 23
Lista_Dwukierunkowa/main.cpp, 24
Lista_Dwukierunkowa/Node.h, 25
Lista_Dwukierunkowa/NodeFactory.cpp, 26
Lista_Dwukierunkowa/NodeFactory.h, 26
ListIterator, 14
    current, 16
    displayCurrent, 15
    ListIterator, 15
    next, 15
    prev, 16
ListIterator.h
    LISTITERATOR_H, 23
LISTITERATOR_H
    ListIterator.h, 23

main
    main.cpp, 24
main.cpp
    main, 24

next
    ListIterator, 15
    Node, 17
Node, 16
    data, 17
    next, 17
    Node, 17
    prev, 17
Node.h
    NODE_H, 25
NODE_H
    Node.h, 25
NodeFactory, 18
    createNode, 18
NodeFactory.h
    NODEFACTORY_H, 26
NODEFACTORY_H
    NodeFactory.h, 26

prev
    ListIterator, 16
    Node, 17

removeAtIndex
    List, 13

removeBack
List, 13
removeFront
List, 13

tail
List, 14